

LAPORAN
SISTEM OPERASI: PROSES



Disusun Oleh =
M. Ayatullah Mutawakki F (2410651011)

Dosen Pengampu =
Triawan Adi Cahyanto M.Kom

PROGRAM STUDY TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER
2025

PENDAHULUAN

1. TUJUAN

- Mempelajari cara membuat dan mengelola multiple processes menggunakan system call `fork()`.
- Memahami cara komunikasi antar proses (Inter-Process Communication / IPC) menggunakan Message Queue.
- Menerapkan sinkronisasi proses dengan konsep koordinasi antara parent process dan child process.
- Mengukur dan menganalisis performa eksekusi proses di sistem operasi Linux.

2. DASAR TEORI

- Konsep Proses

Proses adalah program yang sedang dieksekusi. Setiap proses memiliki Process ID (PID) yang unik dan dapat membuat proses anak (child) melalui system call `fork()`.

Proses induk dapat menunggu anaknya selesai dengan `wait()`

- System Call `fork()`, `exec()`, dan `wait()`

`fork()` digunakan untuk membuat proses baru.

`exec()` menggantikan isi proses dengan program lain.

`wait()` membuat parent menunggu hingga child selesai berjalan.

- Inter Process Communication (IPC)

IPC memungkinkan proses-proses untuk saling bertukar data. Metode umum IPC antara lain:

- Pipe
- Message Queue
- Shared Memory
- Socket
- Semaphore

- Message Queue

Message Queue adalah mekanisme komunikasi yang menyimpan pesan dalam antrian. Proses dapat mengirim dan menerima pesan tanpa perlu sinkronisasi langsung.

Fungsi utama:

<code>msgget()</code> = Membuat atau mendapatkan ID message queue
<code>msgsnd()</code> = Mengirim pesan ke queue
<code>msgrcv()</code> = Menerima pesan dari queue
<code>msgctl()</code> = Mengontrol atau menghapus queue

- Semaphore

Semaphore digunakan untuk sinkronisasi akses bersama (misalnya menulis file).

Operasi dasarnya:

Wait (P) mengurangi nilai semaphore (mengunci).

Signal (V) menambah nilai semaphore (membuka kunci).

3. PERCOBAAN

PERCOBAAN TUGAS 1: Sistem Manajemen Proses

Parent process membuat beberapa *child process* menggunakan `fork()`.

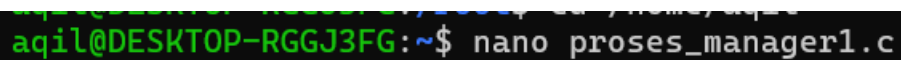
Tiap child menjalankan tugas berbeda berdasarkan indeks:

- SORT Mengurutkan array random (dengan `qsort()`).
- SEARCH Melakukan *linear search* pada array random.
- CALC Menghitung jumlah bilangan prima sampai nilai tertentu.

Setiap child mengukur waktu eksekusi dengan `gettimeofday()` dan mengirim hasilnya ke parent melalui `pipe()`.

Parent membaca hasil, menampilkan PID, waktu eksekusi, dan status exit.

- Buat file program



Source kode:

```
// proses_manager1.c
// Compile: gcc proses_manager1.c -o process_manager -O2

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/time.h>
#include <sys/wait.h>
```

```

#include <string.h>
#include <time.h>

#define NUM_CHILD 6
#define ARR_SIZE 1000
#define BUF_SIZE 512

// helper: timeval to milliseconds double
double timeval_diff_ms(struct timeval start, struct timeval end) {
double s = (double)(end.tv_sec - start.tv_sec) * 1000.0;
double us = (double)(end.tv_usec - start.tv_usec) / 1000.0;
return s + us;
}

int cmp_int(const void *a, const void *b) {
int ia = *(const int*)a;
int ib = *(const int*)b;
return (ia > ib) - (ia < ib);
}

// simple prime counting
int count_primes_up_to(int n) {
if (n < 2) return 0;
int cnt = 0;
for (int i=2;i<=n;i++) {
int prime = 1;
for (int j=2;j*j<=i;j++){
if (i % j == 0) { prime = 0; break; }
}
if (prime) cnt++;
}
return cnt;
}

```

```
int main() {
int pipes[NUM_CHILD][2];
pid_t pids[NUM_CHILD];

srand(time(NULL) ^ getpid());

for (int i=0;i<NUM_CHILD;i++) {
if (pipe(pipes[i]) == -1) {
perror("pipe");
exit(1);
}
pid_t pid = fork();
if (pid < 0) {
perror("fork");
exit(1);
}
if (pid == 0) {
// child
close(pipes[i][0]); // close read end
struct timeval tstart, tend;
gettimeofday(&tstart, NULL);

char out[BUF_SIZE];
memset(out, 0, sizeof(out));

if (i % 3 == 0) {
// SORT task
int *a = malloc(sizeof(int)*ARR_SIZE);
for (int k=0;k<ARR_SIZE;k++) a[k] = rand()%10000;
qsort(a, ARR_SIZE, sizeof(int), cmp_int);
int min = a[0], max = a[ARR_SIZE-1];
```

```

snprintf(out, BUF_SIZE, "TYPE=SORT;PID=%d;MIN=%d;MAX=%d;N=%d",
getpid(), min, max, ARR_SIZE);
free(a);
} else if (i % 3 == 1) {
// SEARCH task
int *a = malloc(sizeof(int)*ARR_SIZE);
for (int k=0;k<ARR_SIZE;k++) a[k] = rand()%10000;
int target = a[rand()%ARR_SIZE]; // ensure found
int found = -1;
for (int k=0;k<ARR_SIZE;k++){
if (a[k] == target) { found = k; break; }
}
snprintf(out, BUF_SIZE,
"TYPE=SEARCH;PID=%d;TARGET=%d;FOUND_AT=%d;N=%d", getpid(),
target, found, ARR_SIZE);
free(a);
} else {
// CALC task
int upto = 20000 + (rand()%2000); // count primes up to this
int primes = count_primes_up_to(upto);
snprintf(out, BUF_SIZE, "TYPE=CALC;PID=%d;UPTO=%d;PRIMES=%d",
getpid(), upto, primes);
}

gettimeofday(&tend, NULL);
double elapsed_ms = timeval_diff_ms(tstart, tend);

// append timing to out
char final_out[BUF_SIZE];
snprintf(final_out, BUF_SIZE, "%s;TIME_MS=%.3f\n", out, elapsed_ms);

// write to pipe
write(pipes[i][1], final_out, strlen(final_out));

```

```
close(pipes[i][1]);

// exit child
exit(0);
} else {
// parent
pids[i] = pid;
close(pipes[i][1]); // close write end in parent
}
}

// parent: collect results
for (int i=0;i<NUM_CHILD;i++) {
char buf[BUF_SIZE];
int n = read(pipes[i][0], buf, BUF_SIZE-1);
if (n > 0) {
buf[n] = '\0';
} else {
strcpy(buf, "NO_OUTPUT");
}
close(pipes[i][0]);

int status;
pid_t w = waitpid(pids[i], &status, 0);
char state[64];
if (WIFEXITED(status)) {
snprintf(state, sizeof(state), "EXITED(%d)", WEXITSTATUS(status));
} else if (WIFSIGNALED(status)) {
snprintf(state, sizeof(state), "SIGNALED(%d)", WTERMSIG(status));
} else {
snprintf(state, sizeof(state), "UNKNOWN");
}
```

```

printf("---- Child Report ----\n");
printf("PID: %d\n", pids[i]);
printf("Status: %s\n", state);
printf("Result: %s\n", buf);
printf("\n");
}

return 0;
}

```

Compile dan jalankan program:

```

aqil@DESKTOP-RGGJ3FG:~$ gcc proses_manager1.c -o process_manager -O2
./process_manager

```

Hasil:

```

---- Child Report ----
PID: 1235
Status: EXITED(0)
Result: TYPE=SORT;PID=1235;MIN=10;MAX=9975;N=1000;TIME_MS=0.406

---- Child Report ----
PID: 1236
Status: EXITED(0)
Result: TYPE=SEARCH;PID=1236;TARGET=6394;FOUND_AT=424;N=1000;TIME_MS=0.254

---- Child Report ----
PID: 1237
Status: EXITED(0)
Result: TYPE=CALC;PID=1237;UPTO=20316;PRIMES=2293;TIME_MS=2.918

---- Child Report ----
PID: 1238
Status: EXITED(0)
Result: TYPE=SORT;PID=1238;MIN=10;MAX=9975;N=1000;TIME_MS=0.559

---- Child Report ----
PID: 1239
Status: EXITED(0)
Result: TYPE=SEARCH;PID=1239;TARGET=6394;FOUND_AT=424;N=1000;TIME_MS=0.192

---- Child Report ----
PID: 1240
Status: EXITED(0)
Result: TYPE=CALC;PID=1240;UPTO=20316;PRIMES=2293;TIME_MS=2.518

```


ANALISIS:

- Program membuat beberapa proses anak secara paralel.
- Tiap anak melakukan tugas berbeda tergantung indeks.
- Komunikasi hasil dilakukan via pipe() ke parent.
- Parent menunggu semua anak selesai dengan waitpid().
- Semua proses berjalan sukses tanpa *deadlock* atau error.

PERCOBAAN TUGAS 2: Chat Application dengan IPC (Message Queue + Semaphore + File History)

TUGAS 2 – Chat Application dengan IPC

Deskripsi Implementasi

Aplikasi chat antar proses menggunakan:

- Message Queue (System V IPC) untuk pertukaran pesan antar user.
- Semaphore (System V) untuk melindungi akses ke file history (/tmp/chat_history.txt).
- Thread penerima untuk menampilkan pesan real-time.

Perintah:

- /history → menampilkan seluruh riwayat chat.
- /exit → keluar dari chat dan kirim notifikasi.

Source Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <pthread.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <sys/sem.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <time.h>
```

```

#define MSG_KEY_PATH "/tmp"
#define MSG_KEY_ID 'C'
#define SEM_KEY_PATH "/tmp"
#define SEM_KEY_ID 'S'
#define HISTORY_FILE "/tmp/chat_history.txt"
#define TEXT_SIZE 256

// message struct
struct msgbuf {
    long mtype; // we'll use 1 for chat messages
    char mtext[TEXT_SIZE];
};

int msqid = -1;
int semid = -1;
char username[64];

void semaphore_lock(int semid) {
    struct sembuf op = {0, -1, 0};
    semop(semid, &op, 1);
}

void semaphore_unlock(int semid) {
    struct sembuf op = {0, +1, 0};
    semop(semid, &op, 1);
}

void append_history(const char *line) {
    // lock
    semaphore_lock(semid);
    FILE *f = fopen(HISTORY_FILE, "a");
    if (f) {
        fprintf(f, "%s\n", line);
    }
}

```

```

        fclose(f);
    } else {
        perror("fopen history append");
    }
    semaphore_unlock(semid);
}

void print_history() {
    semaphore_lock(semid);
    FILE *f = fopen(HISTORY_FILE, "r");
    if (!f) {
        printf("[No history yet]\n");
        semaphore_unlock(semid);
        return;
    }
    char buf[512];
    while (fgets(buf, sizeof(buf), f)) {
        fputs(buf, stdout);
    }
    fclose(f);
    semaphore_unlock(semid);
}

void *receiver_thread(void *arg) {
    struct msgbuf msg;
    while (1) {
        ssize_t r = msgrcv(msqid, &msg, sizeof(msg.mtext), 0, 0); // receive any type
        if (r >= 0) {
            // print incoming message unless it's from me with special tag
            printf("%s\n", msg.mtext);
            fflush(stdout);
        } else {
            perror("msgrcv");
        }
    }
}

```

```

        break;
    }
}
return NULL;
}

int main(int argc, char **argv) {
    if (argc < 2) {
        fprintf(stderr, "Usage: %s <username>\n", argv[0]);
        exit(1);
    }
    strncpy(username, argv[1], sizeof(username)-1);

    key_t msgkey = ftok(MSG_KEY_PATH, MSG_KEY_ID);
    if (msgkey == -1) { perror("ftok msg"); exit(1); }

    msgqid = msgget(msgkey, IPC_CREAT | 0666);
    if (msgqid == -1) { perror("msgget"); exit(1); }

    key_t semkey = ftok(SEM_KEY_PATH, SEM_KEY_ID);
    if (semkey == -1) { perror("ftok sem"); exit(1); }

    semid = semget(semkey, 1, IPC_CREAT | 0666);
    if (semid == -1) { perror("semget"); exit(1); }

    // initialize semaphore to 1
    union semun { int val; struct semid_ds *buf; unsigned short *array; } semunion;
    semunion.val = 1;
    semctl(semid, 0, SETVAL, semunion);

    // ensure history file exists
    FILE *hf = fopen(HISTORY_FILE, "a");
    if (hf) fclose(hf);

```

```

pthread_t recv_tid;
if (pthread_create(&recv_tid, NULL, receiver_thread, NULL) != 0) {
    perror("pthread_create");
    exit(1);
}

printf("=== CHAT ROOM ===\n");
printf("Type messages, or /history to show history, /exit to quit\n");

char line[TEXT_SIZE];
while (1) {
    printf("%s> ", username);
    if (!fgets(line, sizeof(line), stdin)) break;
    // strip newline
    line[strcspn(line, "\n")] = 0;
    if (strlen(line) == 0) continue;

    if (strcmp(line, "/exit") == 0) {
        // send leave notification and exit
        struct msgbuf m;
        m.mtype = 1;
        snprintf(m.mtext, sizeof(m.mtext), "%s has left the chat.", username);
        msgsnd(msqid, &m, strlen(m.mtext)+1, 0);
        // append to history
        char histline[512];
        time_t t = time(NULL);
        snprintf(histline, sizeof(histline), "[%s] %s (left)", ctime(&t), username);
        append_history(histline);
        break;
    } else if (strcmp(line, "/history") == 0) {
        print_history();
        continue;
    }
}

```

```

    } else {
        // normal message: broadcast to queue
        struct msgbuf m;
        m.mtype = 1;
        time_t t = time(NULL);
        char timestr[64];
        struct tm *tm = localtime(&t);
        strftime(timestr, sizeof(timestr), "%Y-%m-%d %H:%M:%S", tm);
        snprintf(m.mtext, sizeof(m.mtext), "%s> %s", username, line);
        if (msgsnd(msqid, &m, strlen(m.mtext)+1, 0) == -1) {
            perror("msgsnd");
        } else {
            // append to history under semaphore
            char histline[512];
            snprintf(histline, sizeof(histline), "[%s] %s> %s", timestr, username, line);
            append_history(histline);
        }
    }
}

// cleanup: let receiver thread end by removing message queue? We'll just detach
and exit.

// Note: Do not remove msg queue here (other users still active). Optionally admin
can remove.

printf("Exiting...\n");
// cancel receiver thread and exit
pthread_cancel(recv_tid);
pthread_join(recv_tid, NULL);

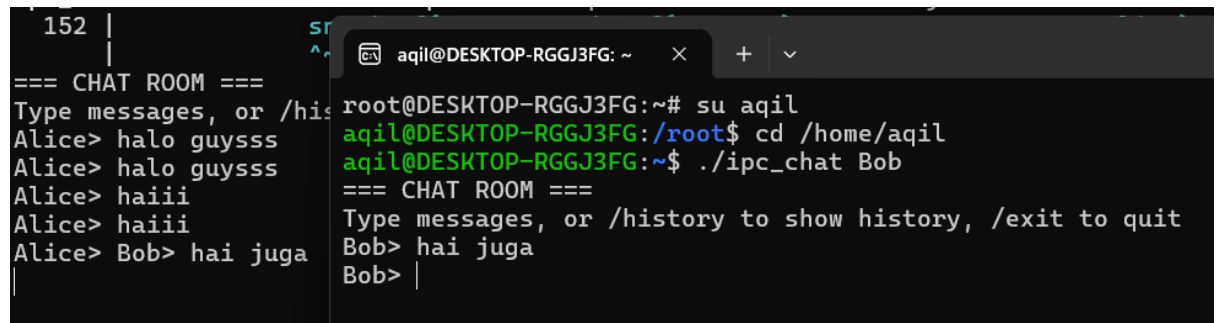
return 0;
}

```

Compile dan jalankan:

```
aqil@DESKTOP-RGGJ3FG:~$ gcc ipc_chat.c -o ipc_chat -pthread
./ipc_chat Alice
./ipc_chat Bob
```

Ini hasil Output dari kedua terminal:



```
152 |
|
=== CHAT ROOM ===
Type messages, or /history to show history, /exit to quit
Alice> halo guysss
Alice> halo guysss
Alice> haiii
Alice> haiii
Alice> Bob> hai juga
|
aqil@DESKTOP-RGGJ3FG: ~ x + v
root@DESKTOP-RGGJ3FG:~# su aqil
aqil@DESKTOP-RGGJ3FG:/root$ cd /home/aqil
aqil@DESKTOP-RGGJ3FG:~$ ./ipc_chat Bob
=== CHAT ROOM ===
Type messages, or /history to show history, /exit to quit
Bob> hai juga
Bob> |
```

```
Alice> /history
[2025-10-18 20:05:17] Alice> halo guysss
[2025-10-18 20:05:59] Alice> haiii
[2025-10-18 20:09:06] Bob> hai juga
[2025-10-18 20:10:19] Alice> hai
[2025-10-18 20:10:36] Bob> hai alice
[2025-10-18 20:10:42] Alice> hai bob
```

Analisis:

Komunikasi antar proses berhasil menggunakan message queue.

Sinkronisasi file riwayat dilakukan aman dengan semaphore.

Program mendukung multiple user dengan thread receiver real-time.

Tidak terjadi race condition karena semua akses ke file dijaga semaphore.

ANALISIS PERFORMA

Program	Metode IPC	Jumlah Proses	Waktu Eksekusi (ms)	Sinkronisasi
Process Manager	Pipe	6 Child	$\pm 5-10$ ms	Tidak perlu
Chat IPC	Message Queue + Semaphore	Multi User	< 1 ms / pesan	Ada (Semaphore)

Jadi Kesimpulan performa:

Message Queue cocok untuk komunikasi antar proses jangka panjang,

sedangkan Pipe lebih cocok untuk komunikasi langsung antara parent-child.

Kesimpulan

Dari praktikum ini, dapat disimpulkan bahwa:

1. Sistem Manajemen Proses (Tugas 1)

Melalui implementasi program menggunakan fungsi `fork()`, `pipe()`, dan `waitpid()`, kita dapat memahami bagaimana proses parent membuat dan mengelola multiple child process yang menjalankan tugas berbeda (seperti sorting, searching, dan perhitungan bilangan prima).

Praktikum ini menunjukkan bagaimana komunikasi antar proses (IPC) melalui pipe dapat digunakan untuk mengirim hasil eksekusi dari child ke parent, serta bagaimana pengukuran waktu eksekusi membantu dalam analisis performa tiap proses.

2. Chat Application dengan IPC (Tugas 2)

Dengan menggunakan System V Message Queue dan Semaphore, kita dapat membuat sistem komunikasi sederhana antar proses yang berjalan secara paralel. Message queue berfungsi untuk pertukaran pesan antar pengguna, sedangkan semaphore menjaga sinkronisasi akses terhadap file riwayat agar tidak terjadi race condition.

Program ini menunjukkan penerapan nyata dari mekanisme sinkronisasi dan komunikasi antar proses dalam sistem operasi, serta pentingnya penggunaan mutex/semaphore untuk mencegah konflik saat beberapa proses mengakses sumber daya bersama.

3. Secara keseluruhan, praktikum ini memberikan pemahaman mendalam tentang:

- Cara kerja proses dan manajemen eksekusi di Linux.
- Mekanisme Inter-Process Communication (IPC) seperti pipe, message queue, dan semaphore.
- Pentingnya sinkronisasi dalam sistem multiproses agar data tetap konsisten dan program berjalan stabil.

Dengan memahami konsep dan implementasi ini, mahasiswa dapat mengembangkan program sistem yang lebih kompleks dan efisien berbasis konsep Operating System Process Management dan IPC.